Guía de estudio — Árboles (Teoría de Grafos y Algoritmos)

Nov 1, 2026 · @Santi

Cómo usar esta guía

Esta guía cubre todo lo de tus apuntes y prácticas de árboles, **menos Huffman** (apunte §5 y ejercicios 13 a 19 de la práctica "Árboles"), que dijiste que no entra. Cada ejercicio está resuelto con el razonamiento completo, para que puedas rehacerlo solo en el examen.

Tema	Qué tenés que saber hacer	Dónde
Árbol y equivalencias	decidir si un grafo es árbol, contar vértices y aristas, bosques	Teoría 1
Terminología con raíz	nivel, altura, padre, hijos, ancestros, descendientes, hojas, internos	Teoría 1
Árbol de expansión	construirlo con DFS y BFS, escribir el orden de las aristas	Teoría 2
Árbol de expansión mínimo	Prim y Kruskal paso a paso, costo total, desempates	Teoría 3
Árboles binarios y m-arios	fórmulas $n = m \cdot i + 1$ y $\ell = (m - 1) \cdot i + 1$, altura, equilibrio	Teoría 4
ABB, recorridos, expresiones	preorden, inorden, postorden, notación polaca	Teoría 5

Orden de estudio sugerido: primero la teoría en orden (cada tema usa el anterior), después la práctica "Árboles con raíz" (ejercicios 1 a 6 son fórmulas, 7 a 12 son recorridos) y por último la práctica "Árboles".

Tres avisos:

- `enunciado.pdf` es el TP integrador de Lógica y Teoría de Grafos (machine learning + grafos, entrega el 08/11/2026). No tiene teoría de árboles, así que no está en esta guía.
- El ejercicio 21 de la práctica "Árboles" dice "G1 a G5", pero el PDF solo trae G1, G2 y G3. Resuelvo esos tres.
- Los grafos de los ejercicios los leí de los dibujos del PDF. Si alguna arista o peso no coincide con tu hoja, avisame y lo rehago.

Teoría 1 — Qué es un árbol y cómo se nombran sus partes

Un árbol es un grafo conexo y sin ciclos: entre dos vértices cualesquiera hay exactamente un camino.

Definición

T = (V, E) es un **árbol** si para todo par de vértices u, r ∈ V existe un **único** camino u–r.

La palabra clave es "único", que esconde dos condiciones a la vez:

- **Existe** al menos un camino entre todo par de vértices: el grafo es **conexo**.
- **No hay dos** caminos distintos entre un mismo par: el grafo no tiene **ciclos**. Si hubiera un ciclo, dos de sus vértices quedarían unidos por dos caminos, uno por cada lado del ciclo.

Por eso "árbol = conexo + acíclico". En el apunte, G1 es árbol, G2 no (tiene un ciclo) y G3 no (no es conexo).

Teorema de equivalencias

Para T = (V, E), estas cuatro afirmaciones son equivalentes:

1. T es un árbol.
2. T es conexo y acíclico.
3. T es simple, conexo y |E| = |V| − 1.
4. T es acíclico y |E| = |V| − 1.

Cómo se usa: para demostrar que un grafo es árbol alcanza con comprobar **dos** propiedades de las tres (conexo, acíclico, |E| = |V| − 1), eligiendo las dos que más fácil se vean. Lo que **no** alcanza es mirar solo la cantidad de aristas: un triángulo más un vértice suelto tiene 4 vértices y 3 aristas (|E| = |V| − 1) y no es árbol, porque no es conexo. Ese es el ejercicio 6 de la práctica.

Respuesta a la pregunta del apunte (¿un árbol con 5 nodos y 6, 5, 4 o 3 aristas?): solo con 4, porque |E| = |V| − 1 = 4. Con 5 o 6 aristas habría ciclos; con 3 aristas un grafo de 5 vértices no puede ser conexo.

Propiedades que se usan en los ejercicios

No figuran como teorema en el apunte, pero salen directo de lo anterior y aparecen en las prácticas:

- **Suma de grados:** la suma de los grados de un árbol es 2|E| = 2(|V| − 1), porque cada arista aporta 1 al grado de cada uno de sus dos extremos.

- **Al menos 2 hojas:** todo árbol con 2 o más vértices tiene como mínimo 2 vértices de grado 1. Razón: ningún vértice tiene grado 0 (es conexo); si hubiera a lo sumo una hoja, los otros n − 1 vértices tendrían grado ⩾ 2 y la suma de grados sería ⩾ 1 + 2(n − 1) = 2n − 1, que es mayor que 2n − 2. Contradicción.
- **Agregar o quitar una arista:** agregar una arista a un árbol crea **exactamente un ciclo**; quitarle una lo deja **desconectado** en dos pedazos.
- **Bosque:** grafo sin ciclos; cada componente conexa es un árbol. Si un bosque tiene n vértices y c componentes, tiene n − c aristas (cada árbol de n_i vértices aporta n_i − 1, y al sumar queda n − c).

Aplicaciones del apunte

La llave de un torneo (semifinales, final, campeón) y un organigrama (director, jefes de departamento) son árboles: el segundo es un árbol con raíz, donde la jerarquía baja desde el director.

Árboles con raíz: terminología

Un **árbol con raíz** es un par (T, r): T es un árbol y r ∈ V es un vértice elegido como raíz. Se dibuja con la raíz arriba. **La raíz es una elección:** el mismo árbol se puede tomar con raíz en distintos vértices (ejercicios 2 y 3 de la práctica "Árboles"); cambian los niveles, los padres y los hijos, pero las aristas son las mismas.

Término	Qué significa
Nivel de u	longitud del camino u–r, o sea la cantidad de **aristas** hasta la raíz. La raíz está en el nivel 0
Altura de (T, r)	el máximo de los niveles de sus vértices
Ancestros de y	todos los vértices del camino r–y, **excepto y** (incluyen a la raíz)
Descendiente	w es descendiente de y si y es ancestro de w
Padre de y	el vértice anterior a y en el camino r–y
Hijo de x	u es hijo de x si x es el padre de u
Hermanos	u y t son hermanos si tienen el mismo padre
Hoja (o terminal)	vértice sin hijos
Vértice interno	vértice que no es hoja, es decir, que tiene hijos

Término	Qué significa
Subárbol de raíz z	el par (T_z, z), donde T_z es el subgrafo inducido por z y todos sus descendientes

Trampas típicas:

- El nivel cuenta aristas, no vértices. Un árbol de un solo vértice tiene altura 0.
- Los ancestros de y **no incluyen a y**. Los descendientes de y tampoco incluyen a y.
- En un árbol sin raíz, "hoja", "vértice colgante" y "terminal" son los vértices de grado 1. En un árbol con raíz, la raíz **no es hoja** aunque tenga un solo hijo: es un vértice interno.

Teoría 2 — Árbol de expansión: DFS y BFS

Un árbol de expansión de un grafo G es un subgrafo que es árbol y contiene todos los vértices de G; existe si y solo si G es conexo.

Definición y propiedad

Un árbol T = (V′, E′) es un **árbol de expansión** (o generador) de G = (V, E) si T es subgrafo de G y contiene a todos sus vértices: V′ = V y E′ ⊆ E.

Propiedad: G tiene un árbol de expansión si y solo si G es conexo.

- Si G tiene un árbol de expansión T, como T es conexo y llega a todos los vértices, G es conexo.
- Si G es conexo y tiene un ciclo, se le saca una arista del ciclo (sigue conexo). Se repite hasta que no queden ciclos, y lo que queda es un árbol con todos los vértices.

Como T es un árbol con |V| vértices, **todo árbol de expansión tiene exactamente |V| − 1 aristas**. Un mismo grafo suele tener varios árboles de expansión distintos (el apunte muestra dos para un grafo de 8 vértices, ambos con 7 aristas).

Qué reciben y qué devuelven DFS y BFS

- **Entrada:** un grafo conexo G y un **orden de los vértices** v1, v2, ..., vn. El orden sirve para desempatar cuando hay varias opciones; si el enunciado no lo da, se usa el orden alfabético o numérico.
- **Salida:** un árbol de expansión con raíz v1.

Construir el árbol equivale a **recorrer** el grafo con un método de búsqueda: cada vez que se llega por primera vez a un vértice, la arista por la que se llegó entra al árbol.

DFS: búsqueda en profundidad (con retroceso)

Idea: avanzar por una rama hasta el final y recién ahí retroceder para probar otra. Usa una **pila** (o recursión).

```
Inicio: V(T) = {v1}, E(T) = vacío, w = v1
Mientras verdadero:
  Mientras exista un vértice vk NO visitado vecino de w:
    elegir ese vk con k mínimo
    agregar la arista (w, vk) a E(T) y vk a V(T)
    w = vk                      // avanzar
  Si w = v1: devolver T
  w = padre de w en T           // retroceder
```

En palabras, paso a paso:

1. Parado en w, mirás sus vecinos que todavía no están en el árbol. Si hay, elegís el de **menor posición en el orden**, agregás la arista y te parás en él.
2. Repetís el paso 1 hasta llegar a un vértice sin vecinos nuevos.
3. Ahí **retrocedés** al padre y volvés al paso 1 desde él. Si el vértice sin vecinos nuevos es la raíz, terminó.

Al escribir "el orden en que se agregan las aristas" se lista solo los avances; los retrocesos no agregan aristas.

BFS: búsqueda en anchura

Idea: primero todos los vecinos de la raíz (nivel 1), después los vecinos nuevos de esos (nivel 2), y así. Usa una **cola**.

```
Inicio: S = (v1), V(T) = {v1}, E(T) = vacío
Mientras verdadero:
  Para cada x de S, en orden:
    Para cada y que no esté en V(T), en orden:
      Si (x, y) es arista: agregar (x, y) a E(T) y y a V(T)
  Si no se agregó ninguna arista: devolver T
  S = los hijos de los nodos de S, en el orden en que se agregaron   // pasar
al nivel siguiente
```

En palabras: se procesa nivel por nivel; dentro de un nivel, de izquierda a derecha; para cada vértice x se agregan **todos** sus vecinos todavía no visitados, de menor a mayor posición en el orden.

Ejemplo del apunte (raíz a, orden a, b, c, d, e, f)

Grafo con aristas ab, ac, bd, cd, cf, de, ef.

- **DFS:** a → b (a tiene vecinos b y c; gana b) → d (el único vecino nuevo de b) → c (d tiene vecinos c y e; gana c) → f → e. Desde e ya no hay vecinos nuevos, así que se retrocede (f, c, d, b) hasta a y termina. Aristas en orden: ab, bd, dc, cf, fe.
- **BFS:** de a salen ab y ac (nivel 1 = b, c). Después b agrega bd, y c agrega cf (nivel 2 = d, f). Por último d agrega de (nivel 3 = e). Aristas en orden: ab, ac, bd, cf, de.

Las aristas que quedaron afuera en cada caso son las que cerrarían un ciclo.

Observaciones del apunte

- Con cualquiera de los dos se puede decidir si un grafo de n vértices es **conexo**: G es conexo si y solo si el árbol T obtenido tiene n vértices.
- En el árbol que construye **BFS**, el nivel de cada vértice es la longitud del **camino más corto** (en aristas) desde la raíz. En DFS no vale eso: el árbol suele quedar largo y delgado, con caminos que no son los más cortos.

	DFS	BFS
Estructura	pila o recursión	cola
Cómo avanza	por una rama hasta el final, después retrocede	por niveles
Forma típica del árbol	largo y delgado	ancho y bajo
Garantiza caminos mínimos desde la raíz	no	sí

Teoría 3 — Árbol de expansión mínimo: Prim y Kruskal

Un árbol de expansión mínimo (AEM) de un grafo ponderado es el árbol de expansión de menor costo total; Prim y Kruskal lo construyen eligiendo siempre la arista más barata que se pueda agregar.

Por qué la solución es un árbol

Problema del apunte: seis ciudades y los costos de construir carreteras entre algunos pares. Se quiere el sistema de carreteras **más barato** que una todas las ciudades. La solución tiene que ser un árbol de expansión por tres razones:

- Debe tener **todos los vértices** (cada ciudad entra en el sistema).
- Debe ser **conexo** (se puede llegar de una ciudad a cualquier otra).

- Debe tener un **único camino simple** entre cada par: si hubiera un ciclo, se podría sacar una arista del ciclo, seguir conectado y gastar menos.

Formalmente: G = (V, E) con un costo c_e > 0 por arista. El costo de un árbol de expansión T es la suma de los costos de sus aristas, c(T) = suma de c_e para e en T, y se busca el T de menor costo.

Ejemplo del apunte (6 ciudades): aristas ab 4, ac 2, bd 4, cd 1, ae 3, ce 6, cf 3, df 6, ef 2. Dos árboles de expansión posibles:

Árbol	Aristas	Costo
T1	ab, bd, ac, ce, cf	4 + 4 + 2 + 6 + 3 = 19
T2	ab, ac, cd, ae, ef	4 + 2 + 1 + 3 + 2 = 12

T2 es el mínimo. Se puede comprobar con Kruskal: cd(1), ac(2), ef(2), ae(3) entran; cf(3) formaría ciclo y se descarta; ab(4) entra. Total 1 + 2 + 2 + 3 + 4 = 12.

Algoritmo de Prim

Idea: se empieza con **un solo vértice** y en cada paso se agrega la arista **más barata que conecte el árbol parcial con un vértice nuevo**. El árbol parcial es siempre un árbol conexo que va creciendo.

```
Entrada: G = (V, E) conexo, costos c_e
Inicio: elegir s en V;  V(T) = {s},  E(T) = vacío
Para i = 1 hasta |V| − 1:
  encontrar (u, v) en E con u en V(T), v fuera de V(T),
    y c(u, v) = mínimo de los costos de las aristas que cruzan
  E(T) = E(T) + {(u, v)};  V(T) = V(T) + {v}
```

Cómo se hace en el examen, paso a paso:

1. Arrancás en el vértice inicial (el primero del orden; "a" si no dicen otro).
2. Escribí las **aristas candidatas**: las que tienen un extremo adentro del árbol y el otro afuera. Las que unen dos vértices ya incluidos **no** son candidatas.
3. Elegí la de menor peso. Si hay empate, aplicá la regla del enunciado (en la práctica: gana la que empieza con la letra menor, ab antes que ac, ac antes que bc).
4. Agregá la arista y el vértice nuevo. Repetí hasta tener |V| − 1 aristas.
5. Sumá los pesos para dar el costo total.

Algoritmo de Kruskal

Idea: se empieza con **todos los vértices y ninguna arista** y se van sumando aristas de menor a mayor peso, saltando las que cierran un ciclo. En el medio hay un **bosque** (grafo sin ciclos) que se va fusionando hasta ser un árbol.

De dónde sale (así lo presenta el apunte): para un bosque de **máximo** costo, el algoritmo goloso agrega en cada iteración la arista de máximo peso que no forma ciclo. Para obtener el AEM se invierten los costos con M = máx c_e y c′_e = M − c_e ⩾ 0. Como todo árbol de expansión tiene |V| − 1 aristas, su costo nuevo es (|V| − 1)·M menos su costo original, así que **maximizar el costo nuevo es lo mismo que minimizar el original**. Al deshacer la inversión queda: en cada iteración se agrega la arista de **mínimo** peso que no forma ciclo con las anteriores.

Cómo se hace en el examen, paso a paso:

1. Ordená todas las aristas de menor a mayor peso (los empates, según la regla del enunciado).
2. Recorré la lista. Para cada arista mirá si sus dos extremos ya están conectados por aristas elegidas: si sí, forma ciclo y se **descarta**; si no, se **agrega**.
3. Pará cuando hay |V| − 1 aristas elegidas.

Algoritmo goloso (greedy)

"Goloso" viene de *greedy* (glotón): construye la solución paso a paso y en cada paso **agarra lo que parece mejor en ese momento**, sin mirar hacia adelante y sin volver atrás para deshacer una elección. Prim y Kruskal son golosos: cada vez eligen la arista más barata posible.

```
Entrada: elementos 1..n con costos c1 >= c2 >= ... >= cn y un criterio de
solución válida
Inicio: F = vacío
Para i = 1 hasta n:
  si F + {i} sigue siendo válida, entonces F = F + {i}   // se queda con lo
mejor disponible y no lo suelta
Devolver F
```

¿El goloso siempre da la solución óptima? No. En algunos problemas, por haber tomado al principio una opción muy buena, ya no se pueden tomar otras que juntas valían más. **Para el árbol de expansión mínimo sí funciona siempre**: la demostración se ve en la práctica (el apunte solo enuncia el teorema).

Prim contra Kruskal

	Prim	Kruskal
Arranca con	un vértice	todos los vértices, sin aristas
Estado intermedio	un árbol conexo que crece	un bosque que se va fusionando
Qué elige	la arista más barata que sale del árbol hacia un vértice nuevo	la arista más barata de todas que no forma ciclo
Chequeo	que el otro extremo esté **fuera** del árbol	que los extremos estén en **componentes distintas**
Necesita vértice inicial	sí	no
Iteraciones	n − 1 agregados (n = cantidad de vértices)	recorre la lista hasta juntar n − 1 aristas

Dos hechos para tener presentes: si hay pesos repetidos puede haber **varios** AEM (todos con el mismo costo total, aunque Prim y Kruskal elijan aristas distintas según el desempate); si todos los pesos son distintos, el AEM es **único**.

Teoría 4 — Árboles con raíz, binarios y m-arios

En un árbol con raíz las aristas "bajan" desde la raíz; si cada vértice interno tiene exactamente m hijos (árbol m-ario completo), la cantidad de vértices, hojas y vértices internos quedan ligadas por fórmulas simples.

Definición precisa de árbol con raíz

- Un **árbol dirigido** es un grafo dirigido cuyo grafo no dirigido asociado (el que queda al borrar las flechas) es un árbol.
- Un árbol dirigido es un **árbol con raíz** si existe un único vértice r, la **raíz**, con grado de entrada ge(r) = 0, y todos los demás vértices v tienen ge(v) = 1. Es decir: las flechas bajan desde la raíz y cada vértice, salvo la raíz, recibe una sola flecha, la de su padre. Por eso se dibuja con la raíz arriba y no hace falta dibujar las flechas.
- El **grado de salida** gs(v) es la cantidad de hijos de v. Si gs(v) = 0, v es una **hoja** (vértice terminal); todos los demás son **vértices internos** (de ramificación).

Ejemplo del apunte: la raíz r tiene hijos n y p; n tiene hijos q y s; p tiene un hijo t; q tiene hijos u y v; s tiene un hijo w; t tiene un hijo x; w tiene hijos y y z. Las hojas son u, v, x, y, z y la altura es 4.

Tipos de árboles con raíz

Nombre	Condición sobre cada vértice v	En palabras
Árbol **binario**	gs(v) = 0, 1 ó 2	a lo sumo 2 hijos, distinguidos como hijo izquierdo e hijo derecho
Árbol binario **completo**	gs(v) = 0 ó 2	cada vértice tiene 2 hijos o ninguno
Árbol **m-ario**	gs(v) ⩽ m	a lo sumo m hijos (con m = 2 es un árbol binario)
Árbol m-ario **completo**	gs(v) = 0 ó m	cada vértice interno tiene exactamente m hijos y ninguna hoja tiene hijos (con m = 2 es el binario completo)

En un árbol binario importa **cuál hijo es el izquierdo y cuál el derecho**: un vértice con un solo hijo da dos árboles distintos según ese hijo sea el izquierdo o el derecho.

Teorema de los árboles m-arios completos

Sea T = (V, E) un árbol m-ario completo con |V| = n vértices, ℓ hojas e i vértices internos. Entonces:

$$n = m\cdot i + 1 \qquad \ell = (m-1)\cdot i + 1 \qquad i = \frac{\ell - 1}{m - 1} = \frac{n - 1}{m}$$

De dónde salen (para poder reconstruirlas en el examen sin memorizar):

1. Todo vértice **excepto la raíz** es hijo de exactamente un vértice interno, y cada interno tiene exactamente m hijos. Entonces los vértices que no son la raíz son m·i, y con la raíz: **n = m·i + 1**.
2. Como cada vértice es hoja o interno, n = i + ℓ. Reemplazando: ℓ = n − i = m·i + 1 − i, o sea **ℓ = (m − 1)·i + 1**.
3. Despejando i de las dos fórmulas: **i = (ℓ − 1)/(m − 1)** y **i = (n − 1)/m**.

Consecuencias para decidir si un árbol existe: n − 1 tiene que ser múltiplo de m, y ℓ − 1 tiene que ser múltiplo de m − 1. Para m = 2: n = 2i + 1 es siempre **impar**, y ℓ = i + 1 (es la propiedad del apunte §9: un binario completo con i internos tiene i + 1 hojas).

Árbol equilibrado

Si h es el nivel máximo de una hoja, el árbol tiene altura h. Un árbol con raíz de altura h es **equilibrado** si el nivel de **cada** hoja es h − 1 ó h. Ejemplos del apunte, ambos de altura 3: el equilibrado tiene hojas solo en los niveles 2 y 3; el no equilibrado tiene una hoja en el nivel 1.

Cota para la altura

Teorema (§9): en un árbol binario con t hojas y altura h vale h ⩾ log2(t).

Por qué: en el nivel k hay a lo sumo 2^k vértices (cada vértice tiene a lo sumo 2 hijos, así que el nivel siguiente a lo sumo duplica). Un árbol de altura h tiene entonces a lo sumo 2^h hojas, o sea t ⩽ 2^h; tomando log2 queda h ⩾ log2(t). Como h es entero, h ⩾ ⌈log2(t)⌉.

Dos cotas que salen de acá y se usan en la práctica, para un árbol binario **completo** de altura h:

- **Máximo de hojas:** 2^h (el árbol "lleno", con todas las hojas en el nivel h).
- **Mínimo de hojas:** h + 1. Hace falta un camino de h aristas hasta la hoja más profunda; cada vértice de ese camino (niveles 0 a h − 1) es interno y tiene 2 hijos, así que aporta una hoja más fuera del camino, más la hoja del final: h + 1.

Teoría 5 — Árboles de búsqueda, recorridos, expresiones y decisión

Un árbol binario se puede recorrer de tres formas (preorden, inorden, postorden) según cuándo se procesa la raíz, y esos recorridos sirven para ordenar datos (ABB) y para escribir expresiones aritméticas sin paréntesis.

Notación de los dibujos en texto de toda la guía: **(I)** = hijo izquierdo, **(D)** = hijo derecho; los hijos van debajo de su padre.

Árbol binario de búsqueda (ABB)

Un **ABB** es un árbol binario donde cada nodo guarda un dato (de un conjunto ordenado: números, palabras…) de modo que para todo nodo v:

- todo dato del **subárbol izquierdo** de v es **menor** que el dato de v, y
- todo dato del **subárbol derecho** de v es **mayor** que el dato de v.

Construcción (inserción). El primer dato queda en la raíz. Cada dato nuevo se compara desde la raíz: si es menor se baja a la izquierda, si es mayor a la derecha, hasta encontrar un lugar vacío, y ahí se coloca como hoja. Ejemplo del apunte, insertando en este orden 40, 25, 60, 10, 30, 70, 65:

```
40
├─ (I) 25
|  ├─ (I) 10
|  └─ (D) 30
└─ (D) 60
   └─ (D) 70
      └─ (I) 65
```

El 65 baja así: 65 > 40 (derecha), 65 > 60 (derecha), 65 < 70 (izquierda, lugar vacío) y queda como hijo izquierdo de 70.

Búsqueda de un dato D. Se empieza en la raíz: si D es igual al dato del nodo, se encontró; si es menor se sigue por el hijo izquierdo; si es mayor, por el derecho. Si se llega a un lugar vacío, D no está en el árbol.

Costo de buscar. En el peor caso el tiempo es proporcional a la **altura**. Cada comparación baja un nivel, así que con altura h hacen falta hasta h + 1 comparaciones. Usando que $h \geq \log_2(t)$ para un binario con t hojas, el peor caso en un ABB con n datos necesita **al menos $\lceil \log_2(n+1) \rceil$ comparaciones**, y ese mínimo se alcanza cuando el árbol tiene altura mínima. Con los 7 datos del ejemplo: $\lceil \log_2 8 \rceil = 3$.

Ojo con el orden de inserción. La forma del árbol depende del orden en que llegan los datos. Si llegan ya ordenados (10, 25, 30, 40, ...), el ABB queda como una cadena y la búsqueda deja de ser rápida.

Los tres recorridos de un árbol binario

Recorrer un árbol es visitar ("procesar") cada nodo **exactamente una vez**, de forma sistemática. DFS y BFS son recorridos; para árboles binarios hay tres recursivos clásicos que solo difieren en **cuándo se procesa la raíz**:

Recorrido	Orden	Cuándo se procesa la raíz
Preorden	raíz, izquierdo, derecho	primero (antes)
Inorden (entreorden)	izquierdo, raíz, derecho	en el medio
Postorden	izquierdo, derecho, raíz	al final (después)

```
preorden(PT):            inorden(PT):             postorden(PT):
  si PT está vacío: volver  si PT está vacío: volver  si PT está vacío:
volver
  procesar PT              inorden(izq de PT)       postorden(izq de PT)
  preorden(izq de PT)      procesar PT              postorden(der de PT)
  preorden(der de PT)      inorden(der de PT)       procesar PT
```

PT es la raíz del (sub)árbol; "procesar" depende del problema (imprimir el dato, sumarlo, evaluarlo…). Siempre se recorre **primero el hijo izquierdo y después el derecho**.

Ejemplo del apunte (este árbol *no* es un ABB: los recorridos se aplican a cualquier árbol binario):

```
H
├─ (I) D
│  ├─ (I) F
│  └─ (D) A
│     └─ (D) I
└─ (D) J
   └─ (D) C
      └─ (I) G
         ├─ (I) B
         └─ (D) E
```

- Preorden: H D F A I J C G B E
- Inorden: F D A I H J B G E C
- Postorden: F I A D B E G C J H

Cómo hacerlo a mano sin perderte. Aplicá la definición recursiva escribiendo, para cada subárbol, "raíz + recorrido del izquierdo + recorrido del derecho" (preorden), "izquierdo + raíz + derecho" (inorden) o "izquierdo + derecho + raíz" (postorden). Un subárbol vacío no aporta nada: un nodo sin hijo izquierdo simplemente salta ese paso.

Propiedad: en un **ABB**, el recorrido **inorden** procesa los datos **ordenados de menor a mayor**. En el ABB de arriba: 10, 25, 30, 40, 60, 65, 70.

Reconstruir un árbol a partir de dos recorridos (lo que piden los ejercicios 9 y 10 de la práctica "Árboles con raíz"):

- El **preorden** dice cuál es la raíz (el primero de la lista); el **postorden** también (el último).
- El **inorden** separa los lados: lo que está antes de la raíz en el inorden es el subárbol izquierdo y lo que está después es el derecho.
- Contando cuántos vértices tiene el lado izquierdo, se sabe qué tramo del preorden (o del postorden) le corresponde, y se repite lo mismo con cada subárbol.
- Con **un solo** recorrido, en general, no alcanza para reconstruir el árbol; por eso se piden dos, o un recorrido más una condición (como la altura).

Árboles de expresión aritmética

Una expresión con operadores binarios + − * / se representa con un árbol binario: las **hojas** son los **operandos** (variables o números) y los **nodos internos** son los **operadores**; cada operador actúa sobre su subárbol izquierdo y su subárbol derecho. La distinción

izquierda/derecha importa: 4 − 6 ≠ 6 − 4. El operador que se aplica **último** queda en la raíz.

Ejemplo del apunte, (C + A) * E − B / D:

```
-
├─ (I) *
|  ├─ (I) +
|  |  ├─ (I) C
|  |  └─ (D) A
|  └─ (D) E
└─ (D) /
   ├─ (I) B
   └─ (D) D
```

Cada recorrido da una forma distinta de escribir la misma expresión:

Recorrido	Forma	Resultado
Inorden (con paréntesis en cada operación)	**infija**	(((C + A) * E) − (B / D))
Preorden	**prefija** o notación polaca	− * + C A E / B D
Postorden	**posfija** o notación polaca inversa	C A + E * B D / −

En las formas prefija y posfija **no hacen falta paréntesis ni reglas de prioridad**: la expresión se evalúa sin ambigüedad. Por eso muchos compiladores traducen a posfija y algunas calculadoras trabajan directamente con ella.

Evaluar una expresión posfija con una pila. Se lee de izquierda a derecha: si el símbolo es un operando, se apila; si es un operador, se desapila primero y y después x, y se apila (x operador y). Al final queda un único valor. Ejemplo C A + E * B D / − con A = 2, B = 8, C = 3, D = 4, E = 5: apilo 3 y 2; el + da 5; apilo 5; el * da 25; apilo 8 y 4; el / da 2; el − da 25 − 2 = **23**.

Evaluar una expresión prefija. Dos maneras equivalentes: (1) recursivamente, un operador va seguido de **dos expresiones completas**, que son sus operandos; (2) leyendo **de derecha a izquierda** con una pila, y al llegar a un operador el primer valor que se desapila es el operando izquierdo y el segundo es el derecho. Por ejemplo, − 5 3 es 5 − 3 = 2.

Árboles de decisión

Un **árbol de decisión** es un árbol con raíz que modela un proceso donde en cada paso se hace una prueba con varios resultados posibles. Cada vértice interno es una prueba (con un hijo por cada resultado) y cada hoja es una conclusión final.

Ejemplo del apunte, la **moneda falsa**: ocho monedas a, b, c, d, e, f, g, h; exactamente una es falsa y más pesada. Hay que encontrarla con una balanza.

- **Árbol binario (altura 3):** se compara un grupo de 4 contra otro de 4. Como hay una sola moneda falsa, los platillos nunca se equilibran: hay dos resultados y cada pesada descarta la mitad de las candidatas (4, luego 2, luego 1). Se necesitan 3 pesadas.
- **Árbol ternario (altura 2):** se compara 3 contra 3 (abc : fgh), dejando 2 monedas afuera. Ahora hay **tres** resultados: baja el platillo izquierdo (la falsa está en abc), baja el derecho (está en fgh) o se equilibran (está en d o e). Con una segunda pesada se la identifica: 2 pesadas.

La cantidad de pesadas del peor caso es la **altura** del árbol. Como cada pesada tiene 3 resultados, con k pesadas se distinguen a lo sumo 3^k casos. Hay 8 monedas que podrían ser la falsa: una pesada no alcanza (3^1 = 3 < 8) pero dos sí (3^2 = 9 ⩾ 8). Con n monedas hacen falta al menos ⌈log3 n⌉ pesadas.

Práctica "Árboles" — ejercicios 1 a 12 resueltos

Estos 12 ejercicios son de introducción: identificar árboles, usar la terminología con raíz, y contar vértices, aristas y grados con el teorema de equivalencias. (Los ejercicios 13 a 19 son de Huffman y no están.)

Ejercicio 1 — ¿Cuáles de los grafos G1 a G4 son árboles?

Criterio: un grafo es árbol si es **conexo y sin ciclos**; o, de forma equivalente, si es conexo y tiene |E| = |V| − 1. Para cada dibujo cuento vértices y aristas, miro si todo está conectado y si hay algún ciclo.

Grafo	Vértices	Aristas	¿Conexo?	¿Ciclos?	¿Árbol?	Explicación
G1	9	8	sí	no	**Sí**	Dos "estrellas" unidas por una arista larga; aristas = vértices − 1 = 8
G2	5	4	sí	no	**Sí**	Un vértice central unido a los otros cuatro; aristas = vértices − 1 = 4
G3	5	5	sí	sí	**No**	Tiene un ciclo (el triángulo de la derecha); hay tantas aristas como vértices, no vértices − 1
G4	5	3	no	no	**No**	Son dos pedazos separados: hay menos aristas que vértices − 1 y no es conexo

Aclaración de lectura: G1 tiene un vértice con 4 vecinos (a la izquierda) unido por una arista larga a otro vértice con 4 vecinos (a la derecha). Si en tu hoja los cuentos dan distinto, rehacé la tabla con el mismo criterio.

Ejercicio 2 — Niveles y altura con raíz en k

Las aristas del árbol son ka, kb, kc, kd, di, ij, de, ef, eg, gh. Nivel de un vértice = cantidad de aristas del camino hasta la raíz k.

```
k
├─ a
├─ b
├─ c
└─ d
   ├─ e
   │  ├─ f
   │  └─ g
   │     └─ h
   └─ i
      └─ j
```

Nivel	Vértices	Cómo se obtiene
0	k	es la raíz
1	a, b, c, d	vecinos directos de k
2	e, i	hijos de d
3	f, g, j	hijos de e (f, g) y de i (j)
4	h	hijo de g: el camino k–d–e–g–h tiene 4 aristas

Altura = 4 (el máximo nivel, el de h).

Ejercicio 3 — El mismo árbol con raíz en d

Ahora d es la raíz. Sus vecinos en el grafo son k, i y e, así que son sus hijos (nivel 1). Después se sigue bajando: k tiene a, b, c; i tiene j; e tiene f y g; g tiene h.

```
d
├─ e
│  ├─ f
│  └─ g
│     └─ h
├─ i
│  └─ j
└─ k
   ├─ a
   ├─ b
   └─ c
```

Nivel	Vértices
0	d
1	e, i, k
2	a, b, c, f, g, j
3	h

Altura = 3. La lección del ejercicio: las **aristas son las mismas**, pero al cambiar la raíz cambian niveles, padres, hijos y altura (con raíz en k la altura era 4; con raíz en d es 3).

Ejercicio 4 — Terminología sobre el árbol T (raíz a)

El árbol: a tiene hijos b, c, d, e; b tiene f; c tiene g, h, i; g tiene m, n; d tiene j; j tiene o, p; e tiene k, l; l tiene q; q tiene r.

```
a
├─ b
│  └─ f
├─ c
│  ├─ g
│  │  ├─ m
│  │  └─ n
│  ├─ h
│  └─ i
├─ d
│  └─ j
│     ├─ o
│     └─ p
└─ e
   ├─ k
   └─ l
      └─ q
         └─ r
```

- **a) Padre de c y de h.** El padre es el vértice inmediatamente arriba. Padre de c: **a**. Padre de h: **c**.
- **b) Ancestros de c y de j.** Son los vértices del camino hasta la raíz, sin contar el propio vértice. De c: **a**. De j: el camino es a–d–j, así que **d y a**.
- **c) Hijos de d y de e.** Los de abajo, directamente conectados. De d: **j**. De e: **k y l**.
- **d) Descendientes de c y de e.** Todo lo que cuelga por debajo, a cualquier profundidad. De c: **g, h, i, m, n**. De e: **k, l, q, r**.
- **e) Hermanos de f y de h.** Mismo padre. f es hijo único de b, así que **no tiene hermanos**. h tiene el padre c, que también tiene a g e i: hermanos de h **g e i**.
- **f) Vértices terminales (hojas).** Los que no tienen hijos: **f, h, i, k, m, n, o, p, r** (9 vértices).
- **g) Vértices internos.** Los que tienen hijos: **a, b, c, d, e, g, j, l, q** (9 vértices).
- **h) Subárbol con raíz en j.** j y todos sus descendientes: j con sus hijos o y p.

```
j
├─ o
└─ p
```

Verificación rápida: 9 hojas + 9 internos = 18 vértices, y un árbol de 18 vértices tiene 17 aristas, que es la cantidad de aristas dibujadas.

Ejercicio 5 — Todos los árboles no isomorfos con 6 vértices

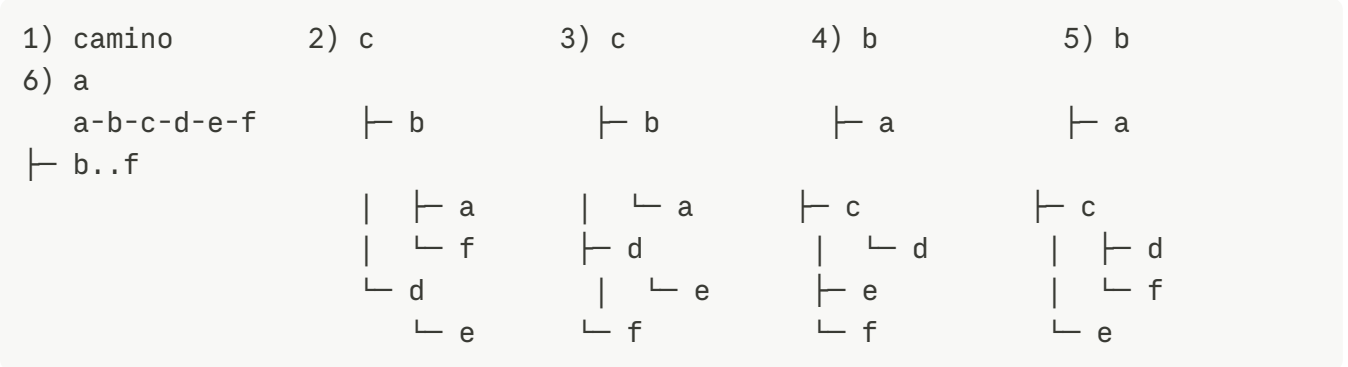
Hay exactamente **6**. Para no olvidarse de ninguno conviene **clasificarlos por la longitud del camino más largo** que contienen (cantidad de aristas):

- **Camino más largo = 5 aristas:** es el camino que pasa por los 6 vértices. Hay 1.
- **Camino más largo = 2 aristas:** todos los vértices cuelgan de uno solo (estrella). Hay 1.
- **Camino más largo = 4 aristas:** un camino de 5 vértices y el sexto vértice colgando de uno de los 3 vértices del medio (si colgara de un extremo, el camino sería más largo). Colgar del 2.º o del 4.º da el mismo árbol por simetría; colgar del 3.º (el central) da otro. Hay 2.
- **Camino más largo = 3 aristas:** un camino de 4 vértices a–b–c–d y dos vértices extra que cuelgan de b o de c: ambos del mismo (b, o c, que es simétrico) o uno de cada uno. Hay 2.

Total: 1 + 1 + 2 + 2 = 6.

N.º	Descripción	Grados	Dibujo (con raíz elegida solo para poder escribirlo)
1	Camino de 6 vértices	2, 2, 2, 2, 1, 1	a–b–c–d–e–f
2	Camino de 5 + vértice colgando del 2.º	3, 2, 2, 1, 1, 1	a–b–c–d–e y f unido a b
3	Camino de 5 + vértice colgando del central	3, 2, 2, 1, 1, 1	a–b–c–d–e y f unido a c
4	Camino de 4 + dos colgando del mismo	4, 2, 1, 1, 1, 1	a–b–c–d, con e y f unidos a b
5	Camino de 4 + uno en cada vértice central	3, 3, 1, 1, 1, 1	a–b–c–d, con e unido a b y f unido a c
6	Estrella	5, 1, 1, 1, 1, 1	a unido a b, c, d, e, f

Los árboles 2 y 3 tienen la misma secuencia de grados pero **no son isomorfos**: en el 2 el vértice de grado 3 tiene **dos** hojas vecinas (a y f); en el 3 tiene **una** sola (f).



Ejercicio 6 — Un grafo con |V| = |E| + 1 que no sea árbol

Ejemplo: un triángulo a–b–c (3 vértices, 3 aristas) más un vértice suelto d. Entonces |V| = 4 y |E| = 3, o sea |V| = |E| + 1, pero tiene un ciclo (el triángulo) y no es conexo (d está aislado), así que no es árbol.

Por qué no contradice el teorema: la igualdad |E| = |V| − 1 caracteriza a los árboles solo **junto con** otra condición (conexo o acíclico). Y de aquí sale algo más fuerte: si un grafo cumple |V| = |E| + 1 y **no** es árbol, tiene que ser a la vez **no conexo y con algún ciclo**; si fuera conexo sería árbol (ítem 3 del teorema), y si fuera acíclico también (ítem 4).

Ejercicio 7 — Cuántos vértices colgantes tiene el árbol

Datos: 4 vértices de grado 2, 1 de grado 3, 2 de grado 4 y 1 de grado 5. Los **colgantes** son los de grado 1. Llamo ℓ a su cantidad (la incógnita).

1. Cantidad de vértices: |V| = 4 + 1 + 2 + 1 + ℓ = 8 + ℓ.
2. Como es árbol: |E| = |V| − 1 = 7 + ℓ.
3. Suma de grados: 4·2 + 1·3 + 2·4 + 1·5 + ℓ·1 = 8 + 3 + 8 + 5 + ℓ = 24 + ℓ.
4. La suma de grados es el doble de las aristas: 24 + ℓ = 2·(7 + ℓ) = 14 + 2ℓ.
5. Despejo: 24 − 14 = 2ℓ − ℓ, o sea $\boldsymbol{\ell = 10}$.

Comprobación: |V| = 18, |E| = 17, suma de grados = 8 + 3 + 8 + 5 + 10 = 34 = 2·17. ✔

Respuesta: 10 vértices colgantes.

Ejercicio 8 — La misma idea en general

Sea v_k la cantidad de vértices de grado k, para k = 2, …, m (los de grado 1, v_1, son la incógnita, como en el ejercicio 7). Hay tres ecuaciones:

1. |V| = v_1 + v_2 + … + v_m.
2. |E| = |V| − 1 (es árbol).
3. v_1 + 2v_2 + 3v_3 + … + m·v_m = 2|E| = 2(|V| − 1).

Reemplazo |V| de la ecuación 1 en la 3: v_1 + 2v_2 + … + m·v_m = 2(v_1 + v_2 + … + v_m) − 2. Paso todo a un lado: v_1 = 2 + v_3 + 2v_4 + … + (m − 2)·v_m. Reemplazando v_1 en la ecuación 1 y agrupando:

```
|V| = 2 + \sum_{k=2}^{m} (k-1)\, v_k \qquad\qquad |E| = 1 + \sum_{k=2}^{m} (k-1)\, v_k
```

Comprobación con el ejercicio 7 (v_2 = 4, v_3 = 1, v_4 = 2, v_5 = 1): la suma da 1·4 + 2·1 + 3·2 + 4·1 = 16, así que |V| = 18 y |E| = 17, igual que antes.

Ejercicio 9 — Probar que todos los árboles son planares

Un grafo es **planar** si se puede dibujar en el plano sin que se crucen aristas. Demostración por **inducción en la cantidad n de vértices**:

- **Base (n = 1):** un solo vértice, sin aristas, se dibuja sin cruces.
- **Paso inductivo:** supongo que todo árbol con n − 1 vértices es planar (n ⩾ 2) y tomo un árbol T con n vértices.
  1. Todo árbol con al menos 2 vértices tiene al menos 2 hojas (Teoría 1). Elijo una hoja v, cuyo único vecino es u.
  2. T − v (sacar v y su única arista) sigue siendo conexo y sin ciclos, o sea es un árbol con n − 1 vértices. Por la hipótesis inductiva tiene un dibujo plano sin cruces.
  3. Pongo v en un lugar libre muy cerca de u y lo uno con u con un segmento corto. Como v tiene **una sola** arista y está pegado a u, esa arista no cruza ninguna otra.
  4. Queda un dibujo plano de T. ∎

Dos justificaciones alternativas, más cortas: (1) **Kuratowski:** un grafo es planar si no contiene una subdivisión de K5 ni de K3,3; ambos contienen ciclos y un árbol no tiene ciclos, así que no puede contenerlos. (2) **Dibujo por niveles:** elegir una raíz y dibujar los hijos de cada vértice debajo de él, sin que las ramas se mezclen.

Ejercicio 10 — Conexo con 30 aristas: máximo de vértices

Todo grafo **conexo** tiene un árbol de expansión, que usa |V| − 1 de sus aristas. Entonces |E| ⩾ |V| − 1, o sea |V| ⩽ |E| + 1 = 30 + 1 = 31. Y se alcanza: un árbol con 31 vértices tiene 30 aristas y es conexo.

Respuesta: |V| máximo = 31.

Ejercicio 11 — Dos árboles T1 y T2

Datos: |E1| = 17 y |V2| = 2|V1|. Los dos son árboles, así que |E| = |V| − 1 en cada uno.

- |V1| = |E1| + 1 = 17 + 1 = **18**.
- |V2| = 2·|V1| = 2·18 = **36**.
- |E2| = |V2| − 1 = 36 − 1 = **35**.

Ejercicio 12 — Bosques

Un bosque con c árboles y n vértices tiene n − c aristas: cada árbol T_i cumple |E_i| = |V_i| − 1, y al sumar los c árboles queda |E| = |V| − c.

- **a)** F1 tiene 7 árboles y |E1| = 40. Como |E1| = |V1| − 7, entonces |V1| = 40 + 7 = **47**.
- **b)** F2 tiene |V2| = 62 y |E2| = 51. Como |E2| = |V2| − c, entonces c = 62 − 51 = **11 árboles**.

Práctica "Árboles" — ejercicios 20 a 22 (expansión, AEM y verdadero/falso)

Acá se aplican DFS, BFS, Prim y Kruskal paso a paso sobre G1, G2 y G3, y se decide si tres afirmaciones sobre árboles de expansión son verdaderas. Todos los resultados numéricos los verifiqué corriéndolos en código.

Los grafos, tal como los leí del PDF

Grafo	Vértices	Aristas con su peso
G1	a, b, c, d, e	ab 3, ad 3, bc 1, cd 2, be 4, ce 1, de 3
G2	a, b, c, d, e, f, g, h, i	ab 4, bc 1, ad 12, be 6, cf 5, bf 7, de 11, ef 14, dg 8, eh 3, fi 9, dh 13, gh 2, hi 10
G3	a, c, e de un lado; b, d, f del otro	ab 2, ad 3, af 3, bc 3, be 1, cd 4, cf 2, de 3, ef 2

Convenciones del enunciado que uso:

- G1 y G2: orden alfabético y raíz (o vértice inicial) **a**.
- G3: orden alfabético **inverso**, o sea f, e, d, c, b, a. El primero del orden es f, así que la raíz (o vértice inicial) es **f**, y entre dos vecinos "el menor" es el que aparece antes en ese orden (e antes que d, d antes que c, y así).
- Desempate de pesos iguales en Prim y Kruskal: gana la arista que "empieza con la letra menor" (ab antes que ac, ac antes que bc). En G3 lo aplico respecto del orden inverso f, e, d, c, b, a, coherente con la regla de que ese es el orden de los vértices. El **costo total** no depende del desempate; lo que puede cambiar es qué aristas se eligen.
- Hay un aviso: el ejercicio 21 pide G1 a G5, pero el PDF solo trae G1, G2 y G3.

Ejercicio 20 — Árbol de expansión con DFS y BFS (se ignoran los pesos)

G1 con DFS, raíz a.

1. En a: vecinos b y d, ninguno visitado. El menor es b. Agrego **ab** y me paro en b.
2. En b: vecinos a (ya está), c y e. Gana c. Agrego **bc**.
3. En c: vecinos b (ya está), d y e. Gana d. Agrego **cd**.
4. En d: vecinos a y c (ya están) y e. Agrego **de**.
5. En e: sus vecinos b, c y d ya están. Retrocedo e → d → c → b → a; en a no queda ningún vecino nuevo y a es la raíz, así que termina.

Orden de aristas: **ab, bc, cd, de** (un camino a–b–c–d–e).

G1 con BFS, raíz a.

1. Proceso a: sus vecinos nuevos, en orden, son b y d. Agrego **ab, ad**. Nivel 1 = (b, d).
2. Proceso b: vecinos a (ya está), c y e. Agrego **bc, be**. Proceso d: sus vecinos a, c y e ya están; no agrego nada. Nivel 2 = (c, e).
3. Proceso c y e: no tienen vecinos nuevos. Termina.

Orden de aristas: **ab, ad, bc, be**.

G2 con DFS, raíz a.

1. a: vecinos b y d. Gana b. **ab**.
2. b: vecinos a, c, e, f. Gana c. **bc**.
3. c: vecinos b y f. Gana f. **cf**.
4. f: vecinos b, c, e, i. Los nuevos son e e i; gana e. **fe**.
5. e: vecinos b, d, f, h. Los nuevos son d y h; gana d. **ed**.
6. d: vecinos a, e, g, h. Los nuevos son g y h; gana g. **dg**.
7. g: vecinos d y h. Gana h. **gh**.
8. h: vecinos d, e, g, i. El único nuevo es i. **hi**.
9. i: sus vecinos f y h ya están. Retrocedo hasta a; en ningún vértice quedan vecinos nuevos y termina.

Orden de aristas: **ab, bc, cf, fe, ed, dg, gh, hi** (8 aristas para 9 vértices; el árbol es un camino).

G2 con BFS, raíz a.

1. a: vecinos nuevos b y d. **ab, ad**. Nivel 1 = (b, d).
2. b: vecinos nuevos c, e, f. **bc, be, bf**. d: vecinos nuevos g y h (a y e ya están). **dg, dh**. Nivel 2 = (c, e, f, g, h).
3. c, e: no tienen vecinos nuevos. f: el nuevo es i. **fi**. g, h: nada nuevo (i ya entró). Nivel 3 = (i).

Orden de aristas: **ab, ad, bc, be, bf, dg, dh, fi**.

G3 con DFS, orden f, e, d, c, b, a, raíz f.

1. f: vecinos a, c, e. En el orden inverso el primero es e. **fe**.
2. e: vecinos b, d, f. Entre b y d gana d (aparece antes en el orden inverso). **ed**.
3. d: vecinos a, c, e. Entre a y c gana c. **dc**.
4. c: vecinos b, d, f. El único nuevo es b. **cb**.
5. b: vecinos a, c, e. El único nuevo es a. **ba**.
6. a: sus vecinos b, d y f ya están. Retrocedo hasta f y termina.

Orden de aristas: **fe, ed, dc, cb, ba**.

G3 con BFS, orden f, e, d, c, b, a, raíz f.

1. f: vecinos nuevos, en el orden del enunciado: e, c, a. **fe, fc, fa**. Nivel 1 = (e, c, a).
2. e: vecinos nuevos d y b, en ese orden. **ed, eb**. c: sus vecinos b, d y f ya están. a: sus vecinos b, d y f ya están. Nivel 2 = (d, b).
3. d y b no tienen vecinos nuevos. Termina.

Orden de aristas: **fe, fc, fa, ed, eb**.

Ejercicio 21 — Árbol de expansión mínimo con Prim y con Kruskal

Recordatorio del método: **Prim** agrega en cada paso la arista más barata que sale del árbol actual hacia un vértice nuevo (las aristas con ambos extremos adentro no cuentan). **Kruskal** recorre todas las aristas de menor a mayor y agrega la que no cierra un ciclo. Los dos paran al tener |V| − 1 aristas.

G1

Prim desde a:

Paso	Árbol actual	Aristas candidatas	Elijo
1	{a}	ab 3, ad 3	ab (empate en 3: ab antes que ad)
2	{a, b}	ad 3, bc 1, be 4	bc 1
3	{a, b, c}	ad 3, cd 2, ce 1, be 4	ce 1
4	{a, b, c, e}	ad 3, cd 2, de 3	cd 2

Aristas en orden: ab, bc, ce, cd. **Costo = 3 + 1 + 1 + 2 = 7.**

Kruskal. Aristas ordenadas (empates por nombre): bc 1, ce 1, cd 2, ab 3, ad 3, de 3, be 4.

Arista	Peso	¿Forma ciclo?	Decisión
bc	1	no	agrego
ce	1	no	agrego
cd	2	no	agrego
ab	3	no (a todavía está suelto)	agrego

Ya hay 4 aristas = |V| − 1, así que paro. Aristas en orden: bc, ce, cd, ab. **Costo = 1 + 1 + 2 + 3 = 7.** Es el mismo árbol que dio Prim.

G2

Acá todos los pesos son distintos, así que no hay empates y el AEM es único.

Prim desde a:

Paso	Árbol actual	Aristas candidatas	Elijo
1	{a}	ab 4, ad 12	ab 4
2	{a, b}	ad 12, bc 1, be 6, bf 7	bc 1
3	{a, b, c}	ad 12, be 6, bf 7, cf 5	cf 5
4	{a, b, c, f}	ad 12, be 6, ef 14, fi 9	be 6
5	{a, b, c, e, f}	ad 12, de 11, eh 3, fi 9	eh 3
6	{a, b, c, e, f, h}	ad 12, de 11, dh 13, gh 2, hi 10, fi 9	gh 2
7	{a, b, c, e, f, g, h}	ad 12, de 11, dh 13, dg 8, hi 10, fi 9	dg 8
8	{a, b, c, d, e, f, g, h}	fi 9, hi 10	fi 9

En el paso 4, bf ya no es candidata porque b y f están los dos adentro. Aristas en orden: ab, bc, cf, be, eh, gh, dg, fi. **Costo = 4 + 1 + 5 + 6 + 3 + 2 + 8 + 9 = 38.**

Kruskal. Aristas ordenadas: bc 1, gh 2, eh 3, ab 4, cf 5, be 6, bf 7, dg 8, fi 9, hi 10, de 11, ad 12, dh 13, ef 14.

Arista	Peso	Componentes de sus extremos	Decisión
bc	1	b y c sueltos	agrego
gh	2	g y h sueltos	agrego
eh	3	e suelto, h en {g, h}	agrego
ab	4	a suelto, b en {b, c}	agrego
cf	5	f suelto, c en {a, b, c}	agrego
be	6	b en {a, b, c, f}, e en {e, g, h}: distintas	agrego (se fusionan)
bf	7	b y f ya están en la misma componente	**descarto** (ciclo)
dg	8	d suelto	agrego

Arista	Peso	Componentes de sus extremos	Decisión
fi	9	i suelto	agrego

Ya hay 8 aristas = |V| − 1: paro. **Costo = 1 + 2 + 3 + 4 + 5 + 6 + 8 + 9 = 38**, el mismo árbol que Prim.

G3 (orden f, e, d, c, b, a)

Prim desde f:

Paso	Árbol actual	Aristas candidatas	Elijo
1	{f}	af 3, cf 2, ef 2	ef 2 (empate con cf: ef gana porque e va antes que c en el orden)
2	{f, e}	af 3, cf 2, be 1, de 3	be 1
3	{f, e, b}	af 3, cf 2, de 3, ab 2, bc 3	cf 2 (empate con ab: gana la que contiene a f, que va primero)
4	{f, e, b, c}	af 3, de 3, ab 2, cd 4	ab 2
5	{f, e, b, c, a}	ad 3, de 3, cd 4	de 3 (empate con ad: gana de, e va antes que a)

Aristas en orden: ef, be, cf, ab, de. **Costo = 2 + 1 + 2 + 2 + 3 = 10.**

Kruskal. Aristas ordenadas (empates según el orden f, e, d, c, b, a): be 1; ef 2, cf 2, ab 2; af 3, de 3, ad 3, bc 3; cd 4.

Arista	Peso	¿Forma ciclo?	Decisión
be	1	no	agrego
ef	2	no (f está suelto)	agrego
cf	2	no (c está suelto)	agrego
ab	2	no (a está suelto)	agrego
af	3	**sí**: a y f ya están conectados (a–b–e–f)	descarto
de	3	no (d está suelto)	agrego

Ya hay 5 aristas = |V| − 1. **Costo = 1 + 2 + 2 + 2 + 3 = 10**, con las mismas aristas que Prim.

Cuidado con los empates en G3. Si el desempate se hiciera con el orden alfabético normal en vez del inverso, Prim saldría con cf, ef, be, ab, ad en lugar de ef, be, cf, ab, de: otro árbol, pero también de costo 10. Eso es normal cuando hay pesos repetidos: puede haber **varios** AEM, todos con el mismo costo.

Ejercicio 22 — ¿Verdadero o falso? (G conexo y ponderado)

a) "Si todos los pesos de G son distintos, los árboles de expansión distintos de G tienen pesos totales distintos." → FALSO

Contraejemplo: dos triángulos que comparten el vértice c. Triángulo a, b, c con ab 1, bc 2, ac 5; triángulo c, d, e con cd 3, de 4, ce 6. Los seis pesos (1 a 6) son distintos.

Un árbol de expansión de este grafo se obtiene sacando **una** arista de cada triángulo (5 vértices necesitan 4 aristas y hay 6). El peso total de todas las aristas es 21, así que el árbol pesa 21 menos lo que se sacó.

- T1 saca bc (2) y cd (3): queda ab, ac, de, ce, con peso 1 + 5 + 4 + 6 = **16**.
- T2 saca ab (1) y de (4): queda bc, ac, cd, ce, con peso 2 + 5 + 3 + 6 = **16**.

T1 y T2 son distintos y pesan lo mismo. Lo que sí es cierto cuando los pesos son todos distintos es que el árbol de expansión **mínimo** es único; pero eso no dice que todos los demás tengan pesos distintos.

b) "Si e es una arista de G cuyo peso es menor que el de cada una de las demás, entonces e está en todo árbol de expansión mínimo." → VERDADERO

Demostración por **intercambio**. Supongo, por absurdo, que T es un AEM que **no** contiene a e.

1. Agrego e a T. Como T es árbol, esto crea **exactamente un ciclo** C, que contiene a e.
2. C tiene alguna otra arista f distinta de e, y f está en T (todas las aristas del ciclo salvo e estaban en T).
3. Saco f: $T' = T + e - f$ sigue conexo y tiene $|V| - 1$ aristas, así que es un árbol de expansión.
4. Su costo es $c(T') = c(T) + c(e) - c(f)$. Como $c(e) < c(f)$ (e es estrictamente más barata que cualquier otra), $c(T') < c(T)$.
5. Eso contradice que T es mínimo. Entonces todo AEM contiene a e. ∎

La palabra "menor" tiene que ser **estricta**. Si dos aristas empatan en ser las más baratas, algún AEM puede dejar una afuera: en un triángulo con los tres pesos iguales a 1, cada AEM omite una de las tres aristas.

c) "Si T es un AEM de G, existe una forma de nombrar los vértices con letras de manera que Prim, recorriendo los vértices en orden alfabético, produce T." → FALSO con la regla de desempate fija de esta práctica

Esta es la más delicada, así que va el razonamiento completo.

Contraejemplo: K4 (4 vértices, todos unidos con todos) con **todos los pesos iguales a 1**. Todo árbol de expansión de K4 tiene 3 aristas y cuesta 3, así que todos son AEM; en particular un camino w–x–y–z es un AEM.

Con la regla del enunciado, Prim siempre arranca en el vértice llamado a, y ante empates gana la arista que empieza con la letra menor:

1. Candidatas: ab, ac, ad (todas pesan 1). Gana **ab**.
2. Candidatas: ac, ad, bc, bd (todas 1). Gana **ac** (empieza con a y su segunda letra es la menor).
3. Candidatas: ad, bd, cd (todas 1). Gana **ad**.

Resultado: la estrella {ab, ac, ad}, centrada en el vértice llamado a. Pongan los nombres como los pongan, Prim produce siempre una **estrella**, nunca un camino. Entonces no existe ningún nombre que haga que Prim produzca el camino T. ∎

Cuándo sí sería verdadera:

- Si todos los pesos son **distintos**, el AEM es único y Prim lo produce sin importar los nombres.
- Si el desempate fuera **libre** (se puede elegir cualquiera de las aristas empatadas), Prim puede producir cualquier AEM T. Razón: en cada paso, sea S el conjunto de vértices ya incluidos y f una arista mínima entre S y el resto. Si f no está en T, al agregar f a T aparece un ciclo que cruza entre S y el resto por otra arista g de T. Entonces T + f − g es un árbol de expansión con costo $c(T) + c(f) - c(g)$; como T es mínimo, $c(g) \leqslant c(f)$, y como f es la más barata que cruza, $c(f) \leqslant c(g)$. O sea $c(g) = c(f)$: g es una arista de T que también es mínima, y Prim puede elegirla.

Para el examen: si el enunciado fija el desempate por letras (como en esta práctica), contestá **falso** con el contraejemplo de K4; si lo marcó como verdadero tu profe, casi seguro está pensando en desempate libre, y conviene aclarar esa hipótesis en la respuesta.